



AI编程与华为云码道概览



前言

- 本章主要讲述编程三次技术变革，详解 Vibe Coding、SDD开发模式与大模型演进历程，探究 AI 赋能编程的新技术逻辑与实操利弊，并通过码道编程工具的介绍，探索使用工具提升 AI 开发能力与效率。



课程目标

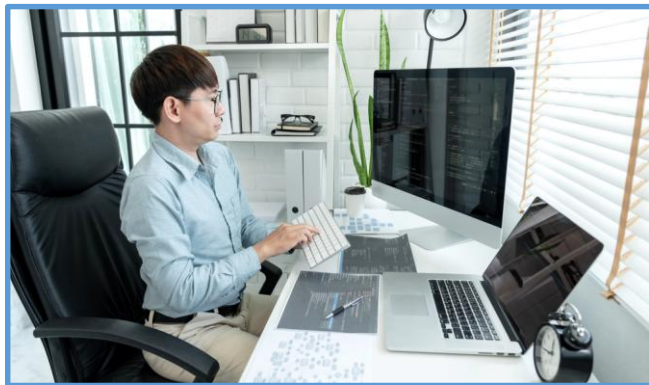
- 学完本课程后，您将能够：
 - 梳理编程三次迭代变革脉络，理清编程范式演进历史
 - 了解 Vibe Coding 编程思路，客观辨识该模式优势与潜在隐患
 - 了解 SDD 规格驱动开发，知道Vibe Coding与SDD的对比
 - 了解华为云码道核心功能
 - 掌握使用码道进行Vibe Coding开发



目录

- 1. 编程世界的三次革命：从打孔纸带到自然语言**
2. Vibe Coding：用直觉写代码的自由与风险
3. SDD：先写说明书，再让AI动工
4. 华为云码道（CodeArts）代码智能体-基础认知
5. 华为云码道（CodeArts）代码智能体-安装与测试

从机器思维到人类语言：编程的三次蜕变



阶段一：汇编语言

1947 - 1950s

汇编符号替代二进制；与硬件强绑定。

MOV

ADD

阶段二：高级语言

1957年起

脱离硬件，拥抱逻辑；平台无关，强大抽象。

Fortran、C、Java、Python

阶段三：自然语言编程

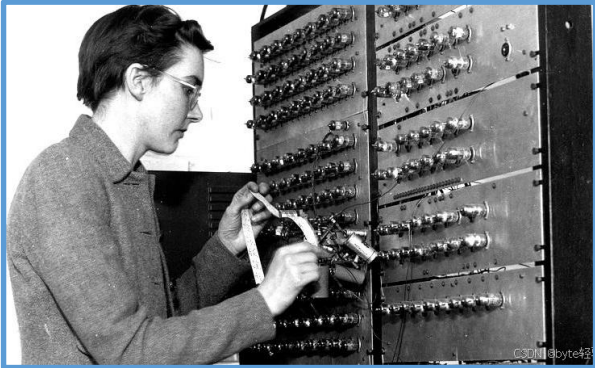
2023年至今

对话即代码，人机协作，需求定义。

“帮我写一个Python脚本，读取这个CSV文件，过滤出销售额大于1万的订单，生成一张按月份统计的柱状图。”

抽象层级的不断跃升，预示着一个「全民皆可创造」的时代正在到来。

第一次革命：汇编语言的出现



凯瑟琳·布斯 (Kathleen Booth)，汇编语言概念的提出者之一，图为其在约1950年操作早期计算机ARC的珍贵影像。

操作行为	机器码示例	汇编指令
数值入累加器	0000 0101	MOV A, #5
寄存器相加	0010 1001	ADD A, B

0和1的机器语言

人与机器的交互建立在冰冷的硬件指令之上，通过打孔卡输入程序。

抽象二进制指令

用助记符来代替复杂的二进制0和1组成的指令。

汇编语言的诞生

确立了“抽象层”的概念，诞生了汇编语言。

从“01代码”到“英语指令”，这不仅是语法的进化，更是人类首次成功“教会”机器理解抽象符号。凯瑟琳·布斯等先驱的工作，让编程从专家的“黑魔法”变成了工程师的实用工具，彻底改变了人类与计算机交互的方式，推动了信息技术革命的第一波浪潮。

第二次革命：高级语言与IDE的出现

高级编程语言是接近人类自然语言、独立于硬件的编程语言，抽象度高、可移植性强，大幅降低开发门槛。

语言	IDE
Python	PyCharm、VS Code、Spyder
Java	IntelliJ IDEA、Eclipse、NetBeans
C++	Visual Studio、CLion
Go	GoLand、VS Code、Vim



IDE 是集成编辑器、编译器、调试器、项目管理的一站式开发工具，显著提升效率。

第三次革命：自然语言编程



开发者：输出自然语言 · 描述业务意图

LLM：新型智能编译器
大模型智能体

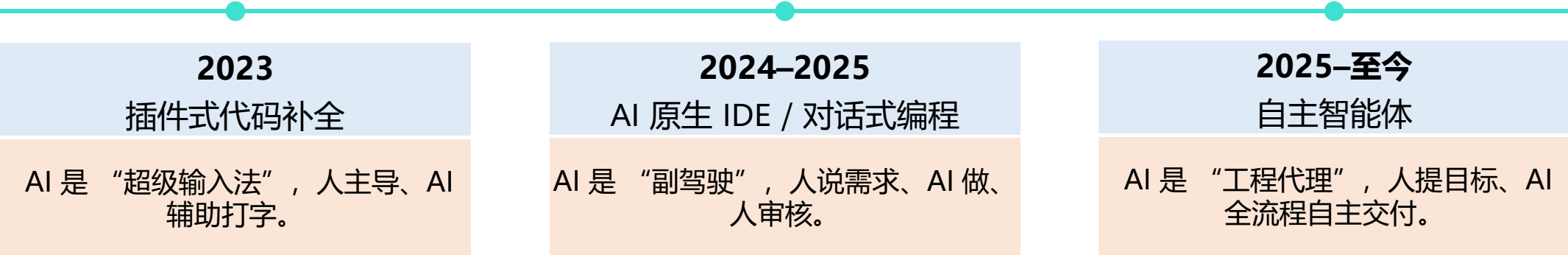
AI 智能体：执行全流程 · 设计 → 编码 → 实现

开发者不再需要手动编写每一行代码，而是通过自然语言向 AI 智能体描述意图，由智能体完成从设计到实现的全过程。

三代编程语言的对比

维度	第一代（汇编时代）	第二代（高级语言时代）	第三代（智能体编排时代）
编程语言	机器码、汇编	C/C++/Java/Python 等	自然语言、结构化规范
核心活动	写指令、管理寄存器	写函数、设计类	写规范、编排智能体
核心技能	理解硬件、位操作	理解数据结构、算法、架构	问题定义、规范编写、智能体驾驭
抽象层级	指令级	函数 / 对象级	意图 / 任务级
人机关系	人直接操控硬件	人写代码，编译器翻译	人定义目标，AI 执行
效率瓶颈	开发效率极低	编码效率低，协作成本高	问题定义精度要求高、质量控制难
工程师定位	硬件操控者	代码生产者	系统价值架构师

AI编程工具的三代演进路径



维度	第一代：插件补全	第二代：AI 原生 IDE	第三代：自主智能体
时间	2023	2024-2025	2025-至今
定位	代码助手（打字）	开发副驾驶（协作）	工程代理（自主交付）
理解粒度	行 / 片段	文件 / 模块	全项目 / 业务
交互方式	键盘 Tab 补全	目标式指令	自然语言对话 + 自主执行
参与流程	仅编码	编码 + 调试 + 重构	需求 → 设计 → 编码 → 测试 → 部署

AI编程工具：第一代 & 第二代



第一代：代码补全助手

2023年以前 · 效率提升阶段

核心能力：超级“Tab键”式补全

依靠代码上下文、Tab 键触发单行 / 代码片段补全

IDE 附属插件

Tab 补全

仅限单文件

依附传统编辑器帮你“补全代码”

IDE + AI插件

输入部分代码

补全剩余片段



第二代：AI-人协同

2024-2025 · 认知协作阶段

核心能力：自然语言交互，人工审核优化

人提需求 → AI 实现 → 人工审核优化，重构开发流程

独立开发环境

自然语言对话

全局项目

AI为独立环境帮你“做出功能”，你来人为审核优化

自然语言需求

AI理解生成代码

修改/调试代码

AI编程工具：第三代：自主智能体代理

目标驱动的自主执行 (2025 至今)

定位：自主工程代理——AI 从 “副驾驶” 升级为可独立交付的工程师，人定目标、AI 全流程自主落地。

代表工具



DeepSeek 开源，让 AI 编码踏入全新发展阶段。

核心特征

目标驱动、全链路自主、多智能体协作

能力边界

需求拆解 → 架构设计 → 编码 → 测试 → 调试 → 部署 → 文档全链路自主完成。

角色转变

开发者 → 智能体调度者 / 架构决策者

人机关系

人定目标 → AI 自主执行 → 人验收，全程无需频繁干预。

软件开发的演进

纯人工主导，开发者全程手动编码

人为主导，AI 做辅助，大部分工作仍需开发者

目标驱动，AI 全链路自主执行，开发者变成架构决策者

1. 传统人工开发阶段

2024年以前

2. AI 辅助开发阶段

2024年-2025年

3. AI 自主智能开发阶段

2025年至今

传统的SDLC 软件开发生命周期

遵循

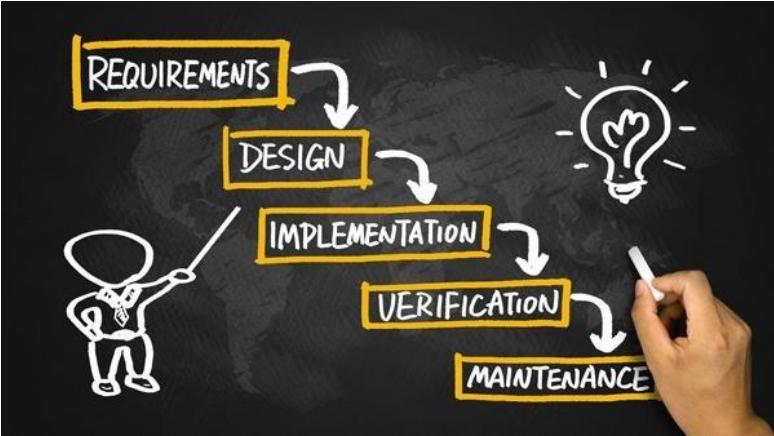
依附

传统的SDLC 软件开发生命周期

诞生

ADLC 智能体开发生命周期

传统SDLC：软件开发生命周期



传统SDLC是什么？

软件开发生命周期，一套人类主导、确定性、阶段式的软件开发标准流程。

它怎么来的？

1960-1970 Winston Royce 提出瀑布模型，成为 SDLC 基础，后续演化出SDLC。

它有什么用？

把混乱开发变成标准化、可管理、可审计流程，降低大型项目风险。

怎么用？ (标准 6 步)

线性推进、阶段不可逆、文档为王、人全程主导

需求分析

定范围、写 SRS (需求规格说明书)

系统设计

架构、模块、数据库、接口设计

开发编码

工程师写代码、单元测试

集成测试

QA 整体测试、Bug 修复

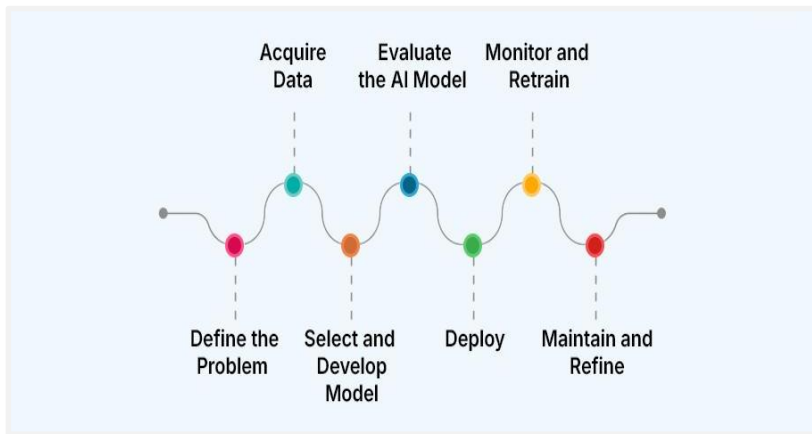
部署上线

发布到生产环境

运维维护

监控、补丁、小迭代

ADLC：智能体开发生命周期



ADLC是什么？

智能体开发生命周期，面向 AI 智能体 / 大模型驱动系统的新型开发范式。

它怎么来的？

传统 SDLC 无法应对 AI 智能体开发与管理，于是诞生了ADLC。

它有什么用？

解决传统 SDLC 在 AI 时代的根本缺陷

怎么用？

(ADLC 核心 5 步 + 闭环)

人定目标、AI 全链路执行、人做决策与风控、系统持续进化

目标锚定

描述自然语言，AI 拆解目标、识别约束

智能体设计

AI 生成架构、角色、提示词、工具链、记忆机制

能力构建

AI 自主写代码、调 API、集成工具、生成测试

仿真评估

AI 自测、边界测试、对齐人类意图、风险评估

动态治理

部署后持续监控、行为分析、模型漂移检测、安全审计

持续反馈闭环

数据与反馈回流，智能体自动迭代优化

SDLC vs ADLC：核心对比

维度	传统 SDLC	ADLC
核心驱动	人主导、代码驱动	目标驱动、AI 智能体主导
行为特性	确定性：同输入→同输出	概率性：同输入→结果可变、可进化
需求形态	早期固定、变更成本高	持续探索、动态调整
流程模式	线性 / 瀑布、阶段不可逆	闭环迭代、持续学习、自主优化
交付方式	一次性交付、版本固定	持续进化、行为在线优化
适用系统	传统软件、需求稳定	AI 智能体、大模型应用、动态复杂系统

研发新范式

机器码 / 汇编

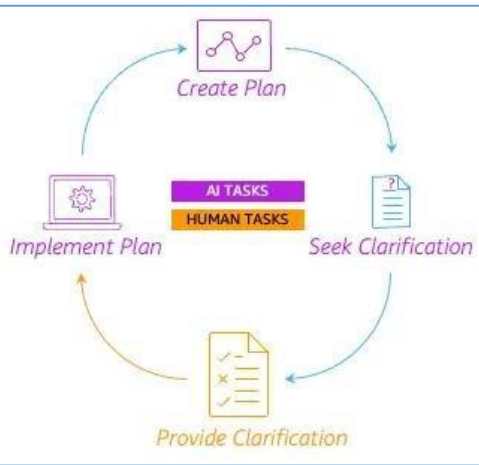
核心：极致性能，零抽象开销

高级语言开发

核心：效率平衡，生态完备

自然语言编程 (AIGC)

核心：极低门槛，极速构建



推荐

ADLC：闭环循环图（目标→设计→构建→评估→治理→反馈回目标）
适用：目标驱动、快速开发、持续进化、AI 主导

不推荐

SDLC：瀑布模型流程图（需求→设计→开发→测试→部署→维护）
适用：传统软件开发、需求稳定项目

未来演进趋势

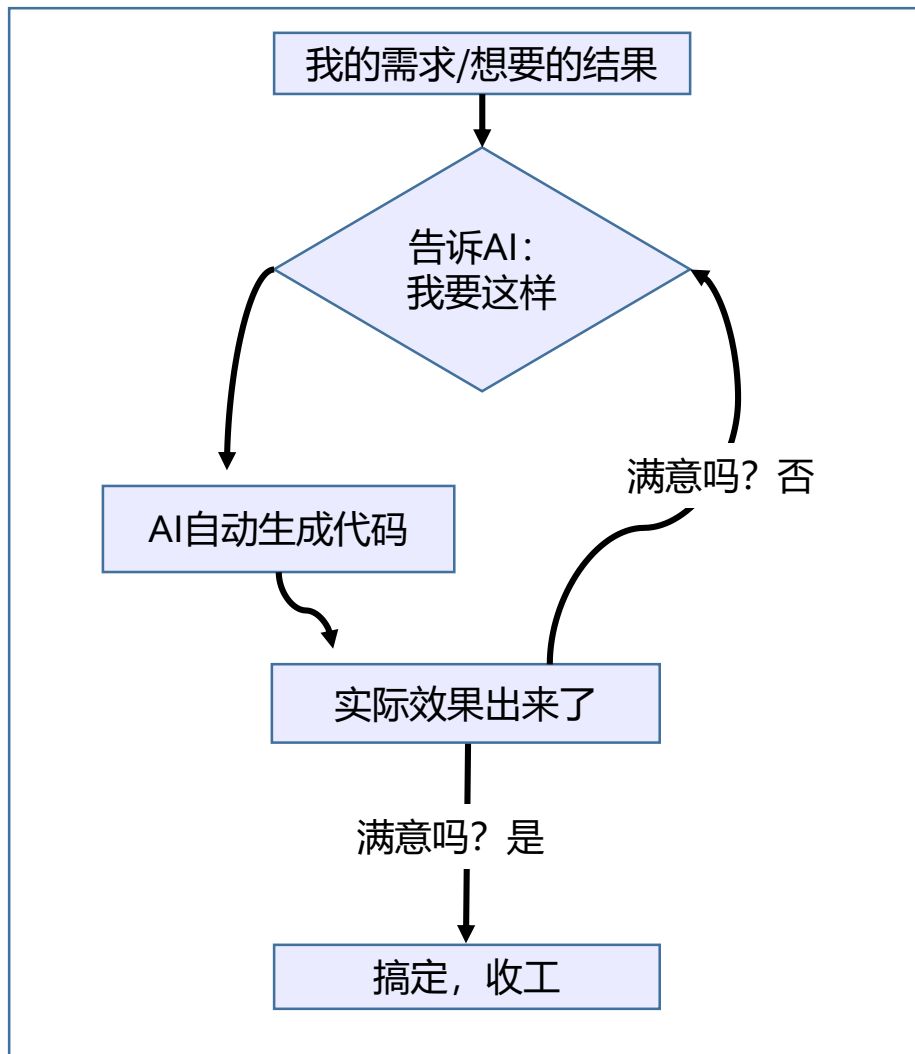
软件开发将走向“底层硬核 + 上层柔性”的深度分层协作。技术发展、AI进化，这已经是未来发展的趋势，拥抱未来实现生产力的跃迁。



目录

1. 编程世界的三次革命：从打孔纸带到自然语言
- 2. Vibe Coding：用直觉写代码的自由与风险**
3. SDD：先写说明书，再让AI动工
4. 华为云码道（CodeArts）代码智能体-基础认知
5. 华为云码道（CodeArts）代码智能体-安装与测试

了解什么是Vibe Coding



Vibe Coding（氛围编程）是一种AI原生的开发范式，开发者通过**自然语言**描述功能意图或“氛围”，无需手写繁琐代码，由AI大模型自动完成代码的生成。



意图描述

自然语言输入
业务需求与场景



AI 引擎

意图精准理解
代码自动生成



对话迭代

通过多轮交互
快速调整效果

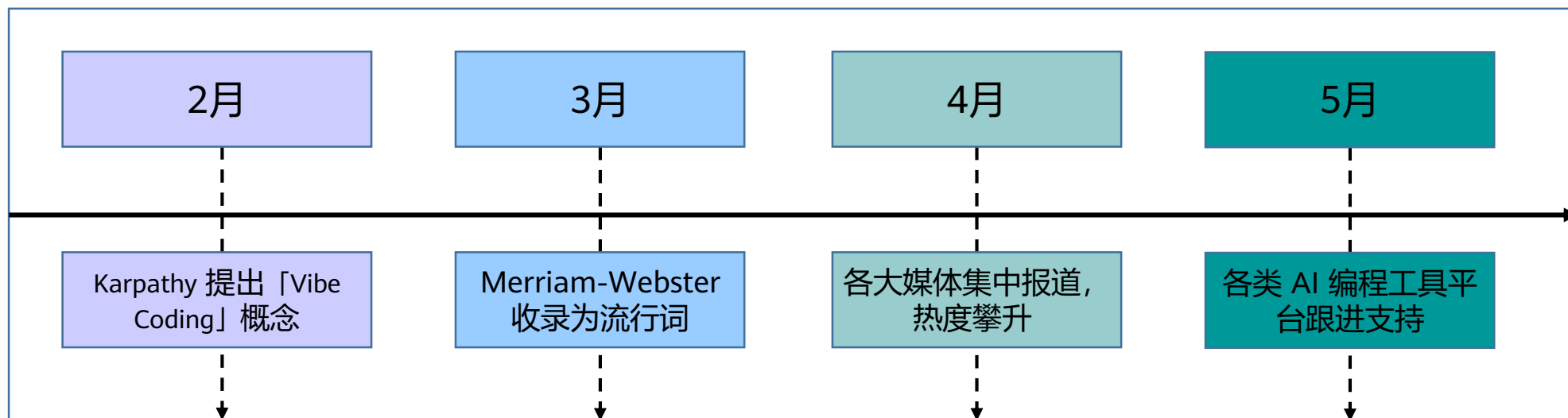


产出：可运行的应用程序

无需编译与配置，直接获得可测试、可部署的成品。从“想法”到“可用产品”的时间被极致压缩，让创新即刻发生。

Vibe Coding的诞生

2025 年 2 月，由Andrej Karpathy（OpenAI 联合创始人、前特斯拉 AI 总监）在 X（原 Twitter）发文正式提出，迅速走红全球，成为 AI 编程新范式。



Vibe Coding 这种新颖的编程范式一经出现便快速传播，成为当下主流的 AI 开发新方式之一。

Vibe Coding有什么用?



核心价值：AI 驱动的意图式编程

开发者的意图 (Vibe) + AI 的生成能力 = 极速落地的代码

打破技术门槛，将编程从“编写指令”升级为“定义价值”，释放核心创造力。

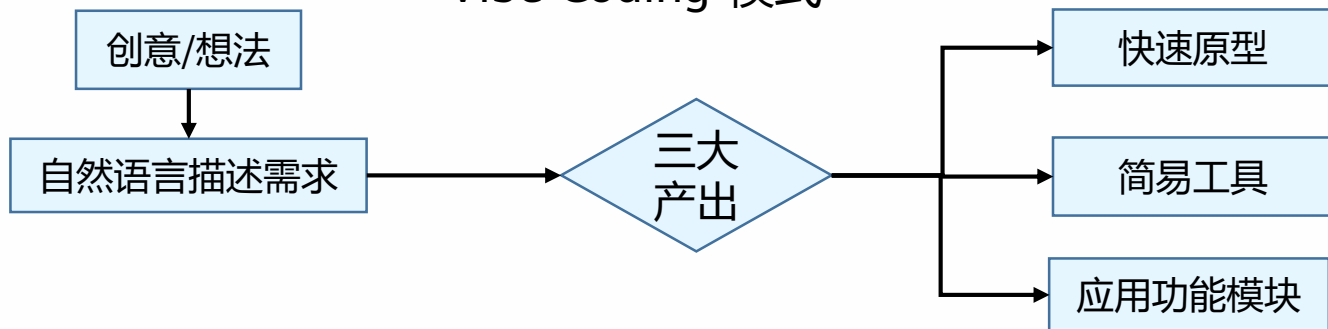
极速提效
缩短周期

降低门槛
全民创作

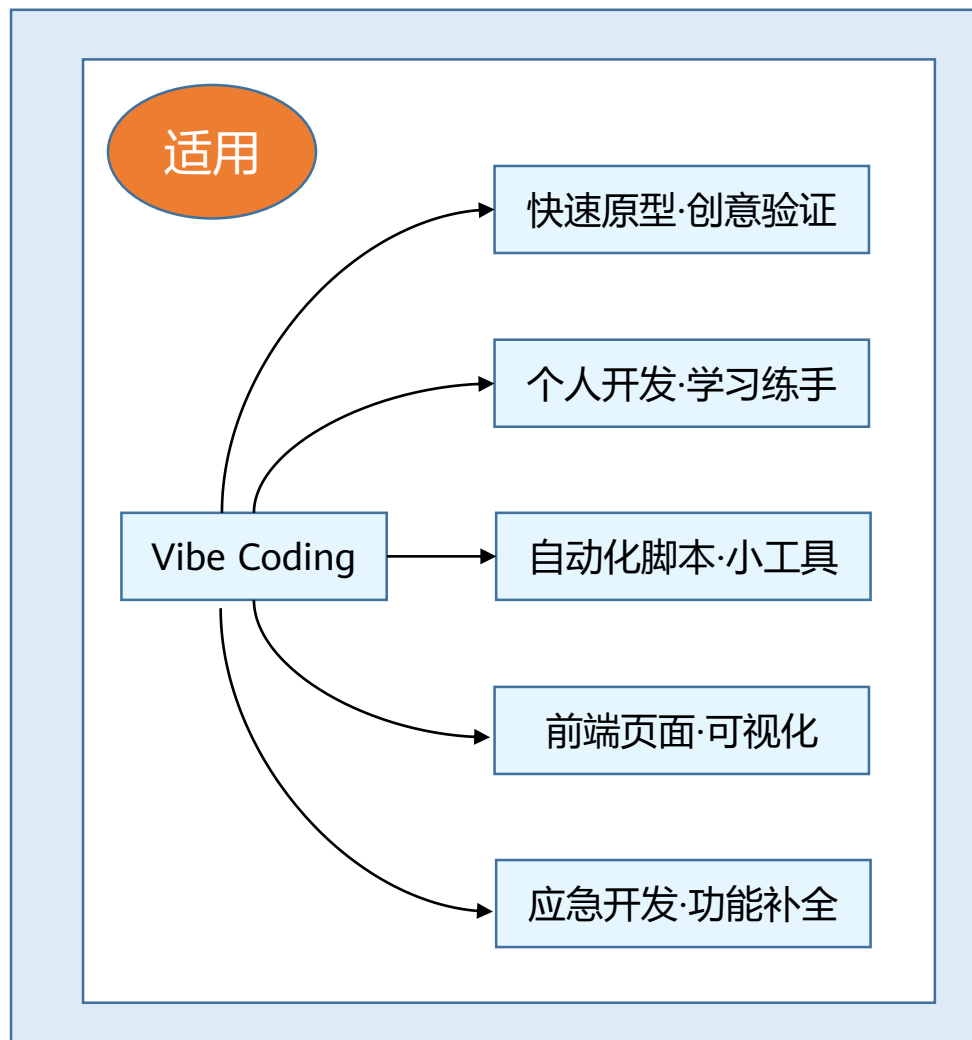
快速试错
验证创意

解放人力
聚焦核心

Vibe Coding 模式

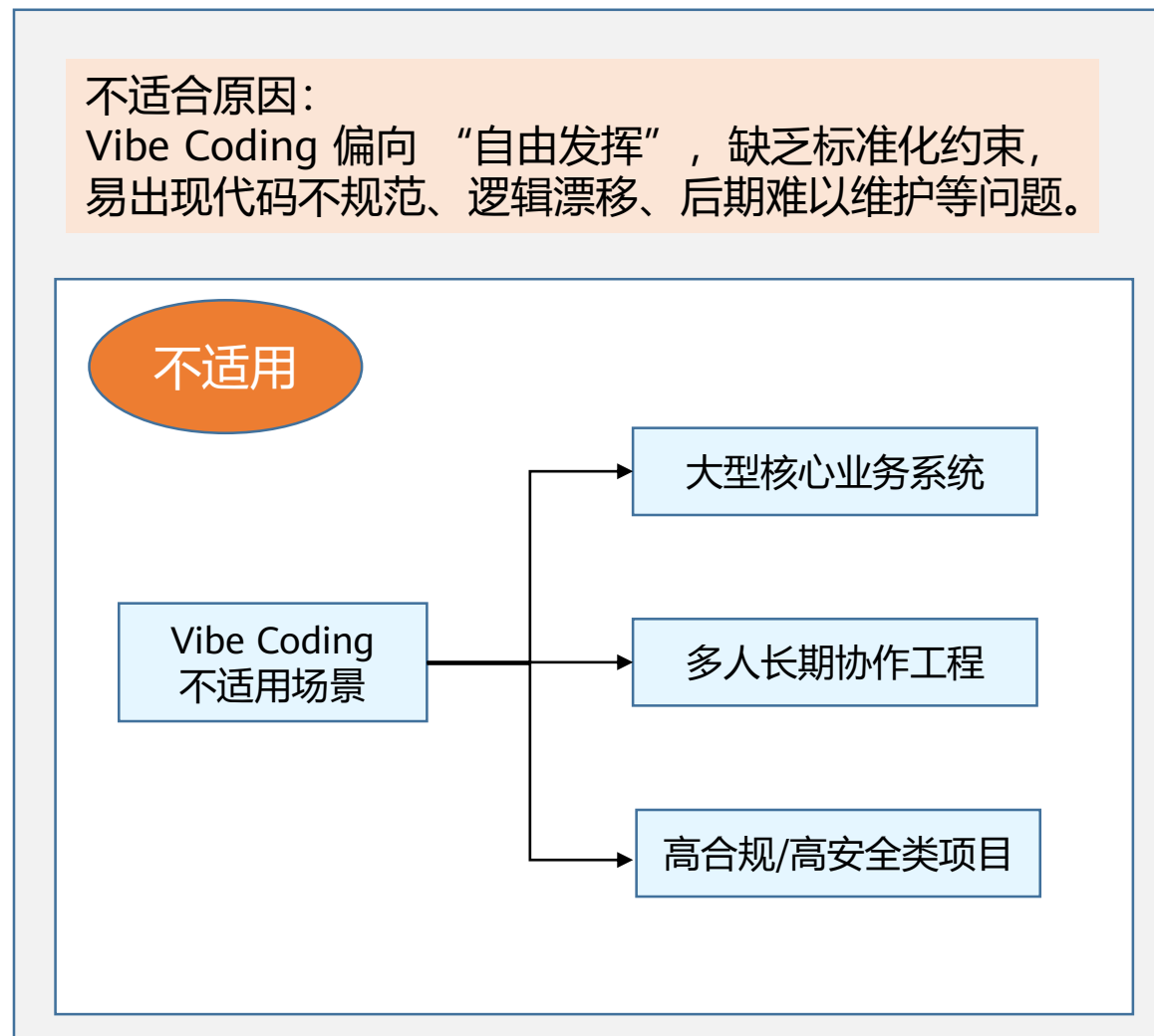


Vibe Coding的适用场景



不适合原因：

Vibe Coding 偏向“自由发挥”，缺乏标准化约束，易出现代码不规范、逻辑漂移、后期难以维护等问题。



Vibe Coding案例

场景描述：

我想快速开发一个西安三日游的前端网页,包含景点、特色美食、路线、酒店等。

```
我来帮你规划西安3日游。首先搜索西安的景点、美食和酒店信息。

mcp_amap-maps-streamableHTTP_maps_text_search

</> ARGUMENTS

{
  "city": "西安",
  "citylimit": true,
  "keywords": "西安 景点 旅游"
}
```

仅口头描述需求，AI 一次性完成界面、数据。



Vibe Coding的风险与局限

1. 代码质量不可控，隐性问题多

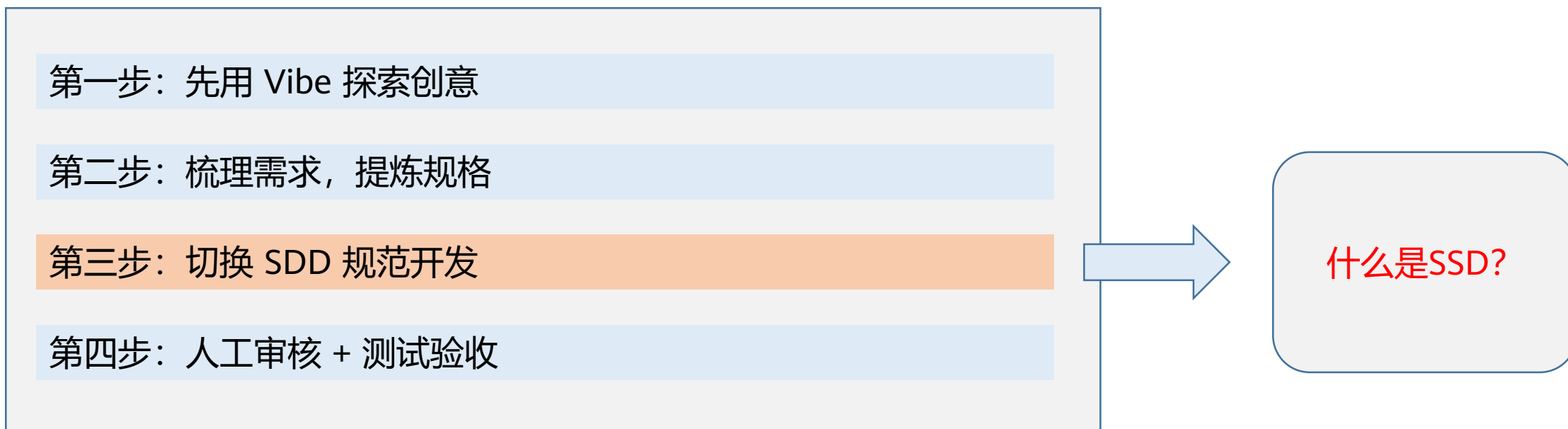
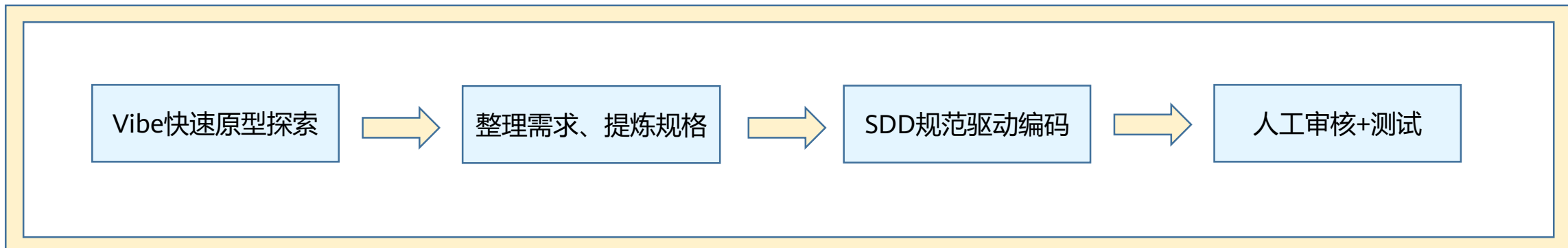
2. 架构混乱，代码难以长期维护

3.需求理解偏差

4. 安全与合规风险



如何高效使用Vibe Coding?





目录

1. 编程世界的三次革命：从打孔纸带到自然语言
2. Vibe Coding：用直觉写代码的自由与风险
- 3. SDD：先写说明书，再让AI动工**
4. 华为云码道（CodeArts）代码智能体-基础认知
5. 华为云码道（CodeArts）代码智能体-安装与测试

SDD是什么？

SDD (Spec-Driven Development, [规格驱动开发](#)) 是一种先写规范、后写代码的 AI 时代软件工程方法论：把 “系统规格 (Spec)” 作为唯一事实来源，由人定 “做什么 / 标准”，AI 负责 “怎么实现”，用规范驱动设计、编码、测试与演进。

核心定义

- 规范先行
- 代码后至
- 规范是契约
- 代码是产物

解决什么问题

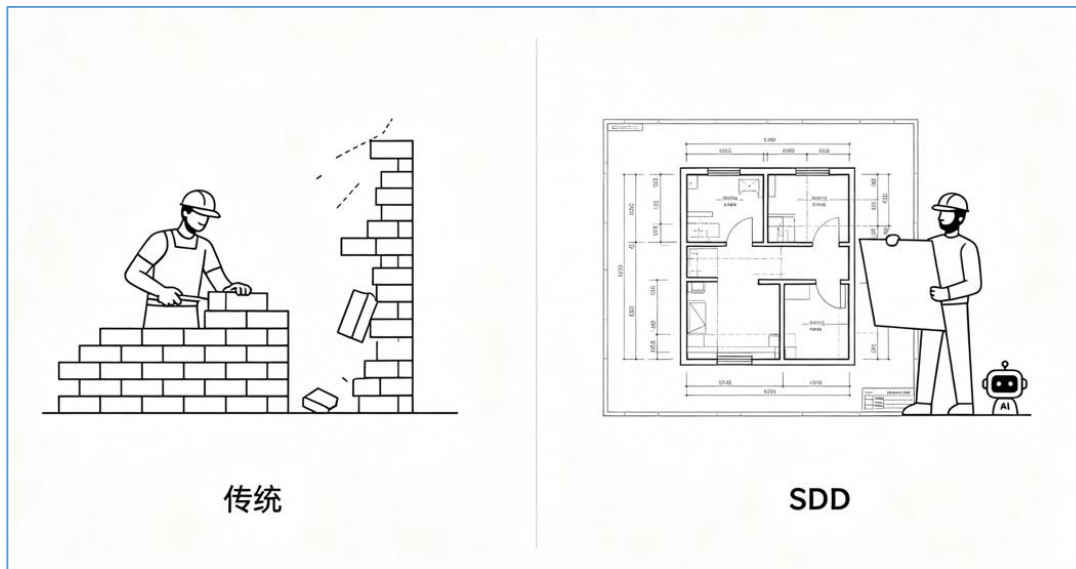
1. 需求模糊、反复返工
2. AI局部正确、全局错误
3. 协作低效、无统一依据
4. 质量滞后、维护成本高

SDD怎么来的？

1980年形式化规范、2000年后出现TDD/BDD

2023-2024 年Vibe Coding 泛滥、AI 代码失控

2025 年，SDD 成为AI 时代软件工程的新范式



TDD（测试驱动）：先写测试、再写代码，侧重单元级正确性

BDD（行为驱动）：填补技术团队与业务方之间的沟通鸿沟

SDD（规格驱动）：先写系统级规格、再计划 / 编码 / 验证，侧重全流程契约与 AI 协作

诞生

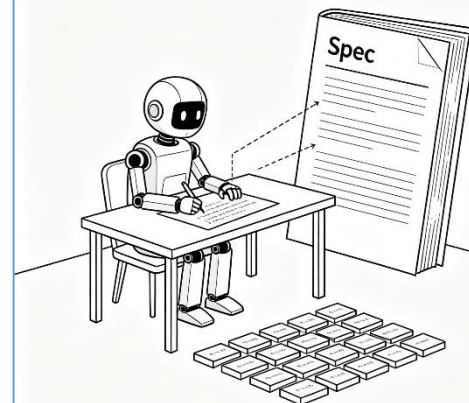
SDD有什么用？

SDD用途1. 约束 AI 编码，解决 Vibe Coding 乱码问题

没有用SDD： AI 凭 prompt 自由发挥，代码架构混乱、私自新增功能。

使用SDD： AI 严格按SDD规范编码，不跑偏、不乱加需求、不漏逻辑。

AI编程中SDD有什么用？



其他
用途

2. 统一协作标准，减少团队沟通内耗

3. 质量左移，提前规避 bug

4. 项目易维护、方便迭代升级

5. 提升落地效率，放大 AI 产能

SDD怎么用?

先写 Spec (规范) → 把Spec发给AI → AI按Spec写代码 → 最后按 Spec 验收

第1步：写Spec (最重要！10分钟搞定)

只写做什么，不写怎么做

- 
1. 要做什么
 2. 输入是什么
 3. 输出必须是什么
 4. 规则/限制
 5. 怎么算通过

第2步：把Spec丢给AI



直接复制发给AI:

AI就不会乱写、不会跑偏、不会漏逻辑。

第3步：按Spec 写代码/开发



不管是人写还是AI写，只遵守Spec，
不随意加功能。

第4步：对照Spec验收



过了 = 完成，没过 = 继续改

先 Spec，后代码；Spec 只写“做什么”，不写“怎么做”；验收只看 Spec，不看感觉。

企业和个人为什么需要SDD



个人开发者为什么用 SDD?

约束 AI 乱生成，避免 AI 凭空加需求、代码结构混乱

提升效率

控制代码

减少返工

个人开发时，先用 Vibe 快速出雏形，SDD 收口规范，兼顾效率与质量。



企业团队为什么用 SDD?

管控 AI 开发风险，统一协作基准，减少线上故障

控风险

控成本

控质量

团队开发时，产品 / 开发 / 测试以 Spec 为唯一标准，依靠 Spec 质量管控，实现项目稳定。

SDD与Vibe Coding的对比

维度	Vibe Coding	SDD
开发顺序	需求 → prompt → 直接生成代码	需求 → 编写 Spec → AI 编码 → 按 Spec 验收
AI 约束力	无约束，AI 自由发挥、擅自加功能	Spec 做契约，AI 严格遵照规格，不能超范围开发
项目规模	适合小型 Demo、临时原型	整体项目、中大型企业正式项目
稳定性	多次生成代码不一致，架构漂移	代码统一对齐 Spec，架构稳定可控
团队协作	口头沟通，理解不一致反复改	全团队共用一份 Spec，统一标准少扯皮
维护交接	无文档，换人看不懂代码	Spec 即项目文档，交接成本极低

SDD + Vibe Coding: 开发新范式



Vibe Coding

(快速生成代码、提升开发效率)

油门 + 提速引擎



SDD (规格驱动开发)

(定规范、控边界、防跑偏)

刹车 + 施工图纸



二者搭配：提速不翻车，开发又快又稳



目录

1. 编程世界的三次革命：从打孔纸带到自然语言
2. Vibe Coding：用直觉写代码的自由与风险
3. SDD：先写说明书，再让AI动工
- 4. 华为云码道（CodeArts）代码智能体-基础认知**
5. 华为云码道（CodeArts）代码智能体-安装与测试

华为云码道（CodeArts）代码智能体-基础认知

什么是华为云码道（CodeArts）代码智能体？

- 华为云码道（CodeArts）代码智能体是华为云打造的智能编码产品，深度融合IDE、自主开发模式与代码大模型能力，支持项目级代码生成、代码续写、研发知识问答、单元测试用例生成等核心功能，可高效提升开发者研发效率，带来优质的智能化编码体验。



华为云码道（CodeArts）代码智能体-基础认知

为什么需要华为云码道（CodeArts）代码智能体？

创新原生编码体验，性能卓越

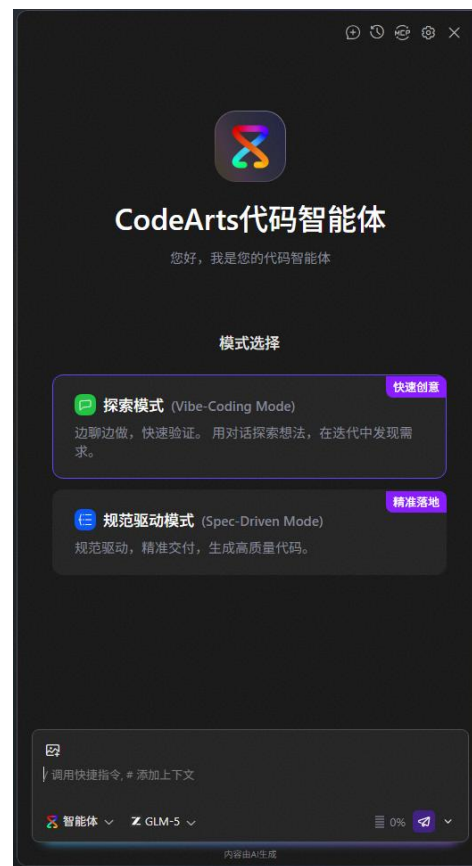
提升应用开发效率
优化资源消耗

平衡开发灵活性与开发规范

探索模式
规范模式

代码库深度理解，代码检索更快更准

项目级深度理解
精准的意图识别



华为云码道（CodeArts）代码智能体-基础认知

华为云码道（CodeArts）代码智能体有哪些功能？

码道的主要功能：

项目级代码生成

单元测试用例生成

代码解释

代码优化

代码续写

代码库索引

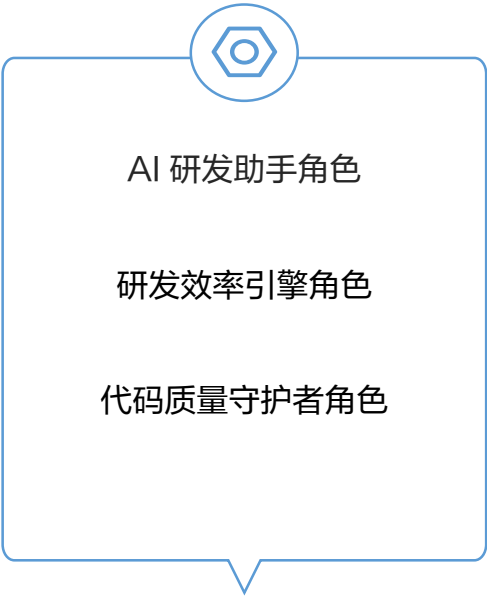
研发知识问答

代码注释

技能 (Skill)

华为云码道（CodeArts）代码智能体-基础认知

华为云码道（CodeArts）代码智能体在现代研发体系中的角色与价值



核心角色



核心价值

对比维度	传统开发 IDE	码道
核心能力	仅提供基础编辑、语法提示，被动辅助编码	智能化主动辅助，支持代码生成、续写、测试生成等全流程能力
质量管控	需手动配置插件，无原生安全与规范检查	原生集成规范、安全检测，自动优化代码质量
效率提升	依赖开发者手动操作，无法实现自动化编码	自动化完成冗余工作，大幅减少手动操作，提升编码效率
学习成本	新手需单独学习插件配置、规范要求，门槛高	内置智能化引导，适配新手，降低技术学习与操作门槛

与传统开发 IDE 对比



目录

1. 编程世界的三次革命：从打孔纸带到自然语言
2. Vibe Coding：用直觉写代码的自由与风险
3. SDD：先写说明书，再让AI动工
4. 华为云码道（CodeArts）代码智能体-基础认知
- 5. 华为云码道（CodeArts）代码智能体-安装与测试**

华为云码道（CodeArts）代码智能体-安装与测试

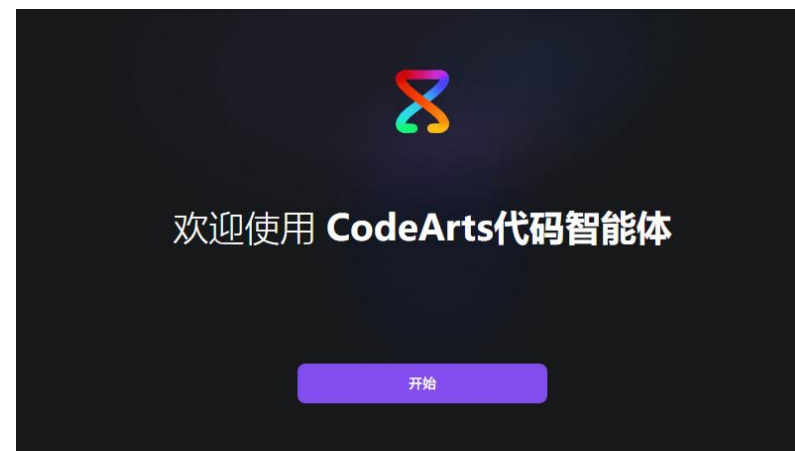
安装并登录华为云码道IDE

操作步骤	说明
准备工作	注册华为账号并开通华为云。
步骤一：安装并登录华为云码道IDE	华为云码道IDE内置了华为云码道插件，您只需下载并安装该IDE，即可直接开始使用相关功能。
	华为云码道IDE当前支持以下平台，本文档以Windows 11（x64）操作系统为例。
	Windows 10（x64）和Windows 11（x64）。
	macOS 11及以上版本，兼容ARM64（Apple Silicon）架构。
	Linux（ARM64）架构，glibc版本 ≥ 2.31，glibcxx版本 ≥ 3.4.26。
步骤二：开发Web应用Demo	在华为云码道中，您只需用自然语言描述需求或功能目标，便能立刻理解，生成结构清晰、工程级的完整代码。
	说明：华为云码道提供了Debug（调试）、Info（信息）、Warn（警告）和Error（错误）四个等级的日志。为方便您后续通过日志排查和定位代码执行中的问题，建议将日志等级设置为Debug以获取更详细的执行信息。如何将设置日志等级，请参见获取日志。
步骤三：效果验证	验证生成的结果是否符合预期。

华为云码道（CodeArts）代码智能体-安装与测试

步骤一：安装并登录华为云码道IDE

- 以Windows 11（x64）操作系统为例，介绍如何安装并登录华为云码道IDE。
- 进入华为云码道（CodeArts）代码智能体官网，下载华为云码道IDE安装包。
- 双击安装程序，根据界面提示完成安装。
- 双击华为云码道IDE图标，启动华为云码道IDE。
- 登录华为云码道IDE。

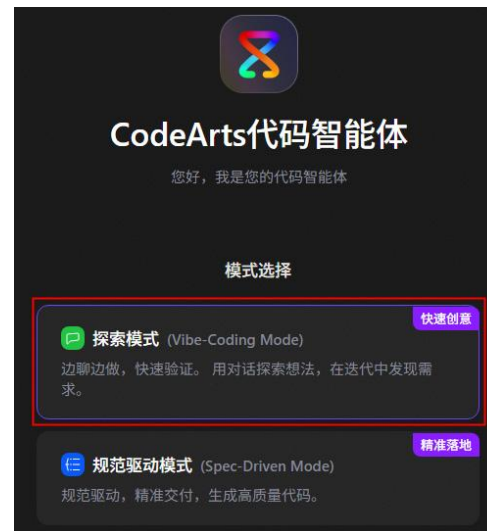
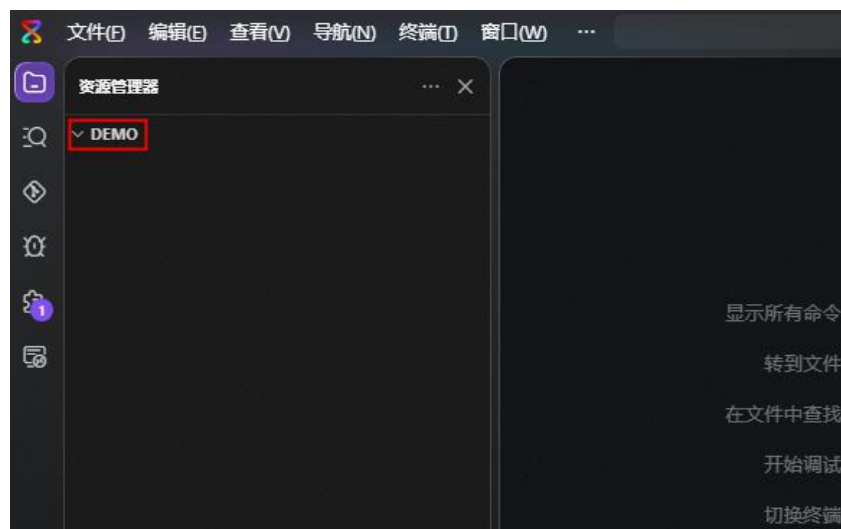


华为云码道（CodeArts）代码智能体官网：<https://codearts.huaweicloud.com/>

华为云码道（CodeArts）代码智能体-安装与测试

步骤二：开发Web应用Demo

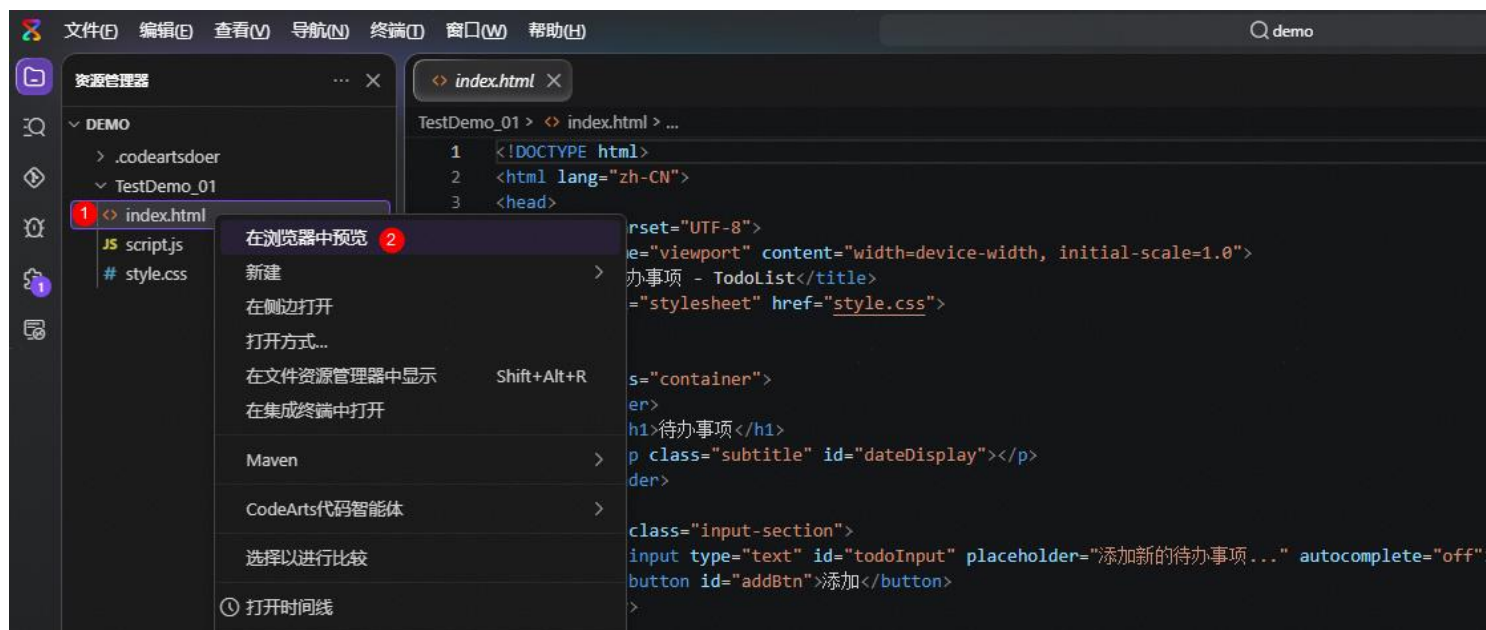
在IDE开发环境中，自主开发模式智能体可直接响应聊天窗口的指令，基于需求描述、编程语言注释或自然语言描述，生成复杂工程级代码。



华为云码道（CodeArts）代码智能体-安装与测试

步骤三：效果验证

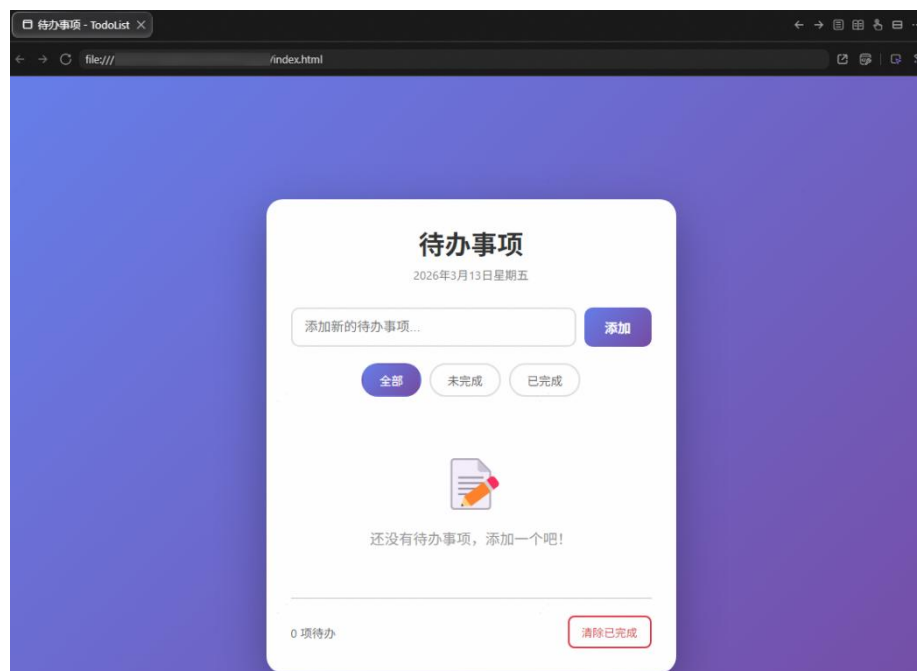
- Web应用创建完成后，在“资源管理器”中展开对应文件夹。将鼠标放在index.html文件上，单击右键选择“在浏览器中预览”。



华为云码道（CodeArts）代码智能体-安装与测试

步骤三：效果验证

- 查看待办事项列表应用，可对Web应用进行功能验证，例如新建、删除及清除已完成事项。

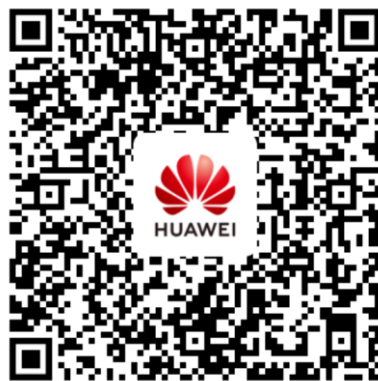




学习推荐

- 华为云官方网站

- 华为云官网: <https://www.huaweicloud.com/>
- 华为云学堂: <https://edu.huaweicloud.com/>



华为云学堂

感谢

版权所有©2026，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编、也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。